BTP Report

Sai Veekshith Kotha

May 13, 2026

1 Project Description

The objective of this project is to develop a system for **Generic Table Extraction from Webpages**. The system aims to automatically identify and extract tabular data from webpages during web scraping, regardless of how the tables are implemented in the frontend code.

Modern webpages often represent tabular information using custom frontend frameworks, dynamic rendering, CSS-based layouts, accessibility tags, or component-based architectures instead of traditional HTML `<table>` tags. As a result, conventional table extraction methods that rely only on standard table elements are insufficient.

This project focuses on building a generalized extraction pipeline capable of detecting and processing both traditional HTML table structures and non-trivial table implementations generated through webpage code. The proposed system aims to identify and extract tabular data from webpage source code and rendered DOM structures irrespective of how the table is internally represented, including div-based layouts, ARIA-based structures, JavaScript-rendered components, and framework-generated tables from technologies such as Angular, React, and Vue.

The project specifically focuses on extracting tables that are code-generated within webpages and does not target tables embedded inside images, screenshots, scanned documents, or PDFs. The goal is to create a scalable and reusable solution capable of extracting structured tabular information from heterogeneous web environments without relying on manually designed website-specific extraction rules.

2 Approaches

The initial phase of the project focused on extracting tables that are explicitly represented through webpage semantics. The goal was to first handle structured table representations before moving towards more generalized non-trivial table extraction.

2.1 Approach 1: HTML Table Based Extraction

The first approach focused on extracting traditional HTML tables using tags such as `<table>`, `<tr>`, `<td>`, `<th>`, `<thead>`, and `<tbody>`.

The implemented pipeline parsed webpage HTML using BeautifulSoup, detected table elements, and extracted headers and row data. A grid-based reconstruction algorithm was also implemented to correctly handle complex structures involving `rowspan` and `colspan`.

This approach successfully handled:

- Standard HTML tables.
- Tables with merged rows and columns.
- Multi-level headers.
- Partially malformed table structures.

However, the method was limited to webpages that explicitly used HTML table semantics.

2.2 Approach 2: ARIA Role Based Extraction

The second approach focused on extracting tables represented using ARIA accessibility roles such as `role="table"`, `role="grid"`, `role="row"`, and `role="cell"`.

Since many modern web applications dynamically generate accessible table-like layouts using frontend frameworks, Selenium was used to access the rendered DOM and identify ARIA-based table structures.

The pipeline performed:

- Discovery of ARIA table and grid containers.
- Extraction of rows and cells using ARIA relationships.
- Header detection and structural validation.
- Handling of dynamically rendered content.

This approach improved support for modern framework-generated tables and accessibility-oriented layouts. However, it still relied on explicit semantic annotations and could not handle fully non-semantic visual table structures.

2.3 Approach 3: Div-Based and Non-Semantic Table Extraction

The next approach focused on extracting tables that are not represented using standard HTML table tags or ARIA roles. Many modern webpages visually display tabular information using nested `div` elements, CSS grid layouts, flexbox structures, or framework-generated components without providing explicit table semantics.

To handle such cases, an experimental extraction pipeline was developed that attempted to identify table-like patterns from the webpage structure and rendered layout. The approach involved analyzing DOM hierarchies, repeated element patterns, alignment consistency, and structural similarities between sibling elements.

The method attempted to detect:

- Div-based table layouts.
- CSS grid and flexbox table-like structures.
- Framework-generated visual tables without semantic annotations.
- Repeated row-column patterns in rendered webpages.

Although the approach worked for certain webpages, it did not perform consistently across different websites. Since non-semantic tables do not follow fixed structural rules, many layouts were difficult to generalize reliably. Variations in frontend design, nested containers, dynamic rendering, and inconsistent visual organization introduced significant challenges in accurately identifying rows, columns, and cell boundaries.

This highlighted the complexity of generalized non-trivial table extraction and motivated the need for more robust structural and visual analysis techniques in future stages of the project.

2.4 Dataset Collection and Large-Scale Extraction

After the initial extraction approaches, the next stage of the project focused on collecting a large-scale dataset of non-semantic web tables. However, this proved to be challenging because div-based and framework-generated tables are difficult to identify automatically during web scraping.

Unlike traditional HTML tables, non-semantic tables do not follow fixed structural rules and are often embedded within complex webpage layouts. To address this, an experimental scalable extraction pipeline was developed using lightweight DOM analysis and unsupervised machine learning techniques instead of relying entirely on rendering-based methods.

The pipeline included:

- Heuristic filtering for identifying potential table-like webpage blocks.
- Feature extraction based on DOM structure and content patterns.
- K-Means clustering for separating probable table structures from non-table content.
- KNN-based classification for faster large-scale extraction.

The approach showed partial success on certain webpages and helped automate parts of the dataset collection process. However, due to the highly heterogeneous nature of modern web layouts, the method did not generalize consistently across different websites. These experiments highlighted the complexity of generalized non-semantic table extraction and identified important challenges for future improvements.

3 Study on LLM Pre-Training Datasets

As a separate study, an analysis was conducted on large-scale pre-training datasets used in modern Large Language Models (LLMs). The study focused on understanding the evolution of dataset scaling, domain composition, multilingual representation, dataset curation methodologies, and the increasing role of synthetic data in modern AI training pipelines.

The study also explored topics such as dataset filtering and deduplication techniques, reasoning-oriented data mixtures, multilingual and regional dataset challenges, and the limitations associated with large-scale web data collection. Additionally, constraints related to data quality, evaluation contamination, scalability, and copyright concerns were examined.

Overall, the study provided a broader understanding of how modern AI systems rely on large-scale data engineering and carefully curated training datasets for improving model performance and reasoning capabilities.